

EX.NO: 6	STRING OPERATIONS STRING CLASS
DATE:	

AIM:

To write a java program to perform string operation using string class

ALGORITHM:

Step 1: Start

Step 2: Input a string from the user

Step 3: Display the input string and its length

Step 4: Input a substring to search

Step 5: Check if the input string contains the substring

Step 6: Display the result (found or not found)

Step 7: Input start and end indices

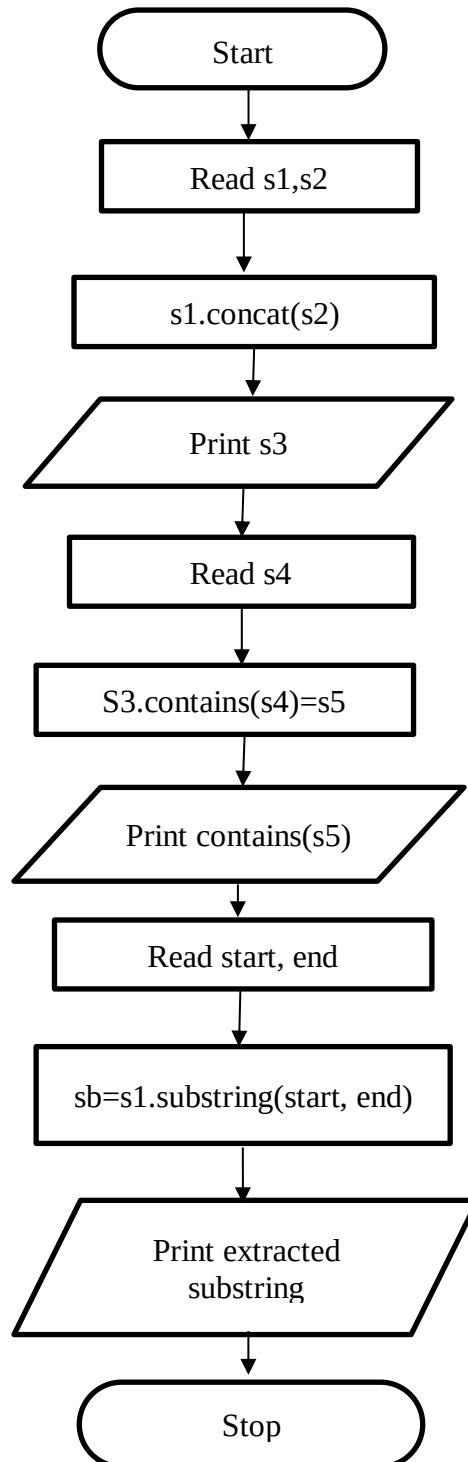
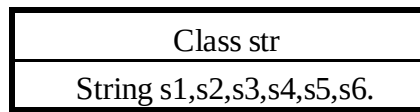
Step 8: Extract a substring from the input string using the start and end indices

Step 9: Display the extracted substring

Step 10: Stop

CLASS DIAGRAM:

FLOWCHART



PROGRAM:

```
import java.util.*;

public class str
{
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.println("Enter the first string:");
        String s1 = s.nextLine();
        System.out.println("Enter the second string:");
        String s2 = s.nextLine();
        String s3 = s1.concat(s2);
        System.out.println("Concatenated String: " + s3);
        System.out.println("Main string: " + s1);
        System.out.println("Enter the substring:");
        String s4 = s.nextLine();
        boolean s5 = s3.contains(s4);
        System.out.println("Substring found? " + s5);
        System.out.println("Main string for substring extraction: " + s1);
        System.out.println("Enter the starting index:");
        int start = s.nextInt();
        System.out.println("Enter the ending index:");
        int end = s.nextInt();
        String s6 = s1.substring(start, end);
        System.out.println("Extracted Substring: " + s6);
        s.close();
    }
}
```

OUTPUT:

Enter the first string: java
Enter the second string: programming
Concatenated String: javaprogramming
Main string: java
Enter the substring: av

Substring found? true
Main string for substring extraction: java
Enter the starting index:
1
Enter the ending index:
4
Extracted Substring: ava

RESULT :

Thus the program has been executed successfully.

EX.NO: 7	STRING OPERATIONS USING STRING BUFFER
DATE:	

AIM:

To write a java program using operation using string buffer class.

ALGORITHM:

Step 1: Start program

Step 2: Create Scanner for input

Step 3: Read string from user and create StringBuffer object

Step 4: Display length using length() method

Step 5: Reverse string using reverse() method and display

Step 6: Reverse back to original (sb.reverse())

Step 7: Read start and end indices for deletion

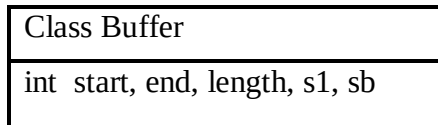
Step 8: Delete characters from start to end-1 using delete(start, end)

Step 9: Display modified string

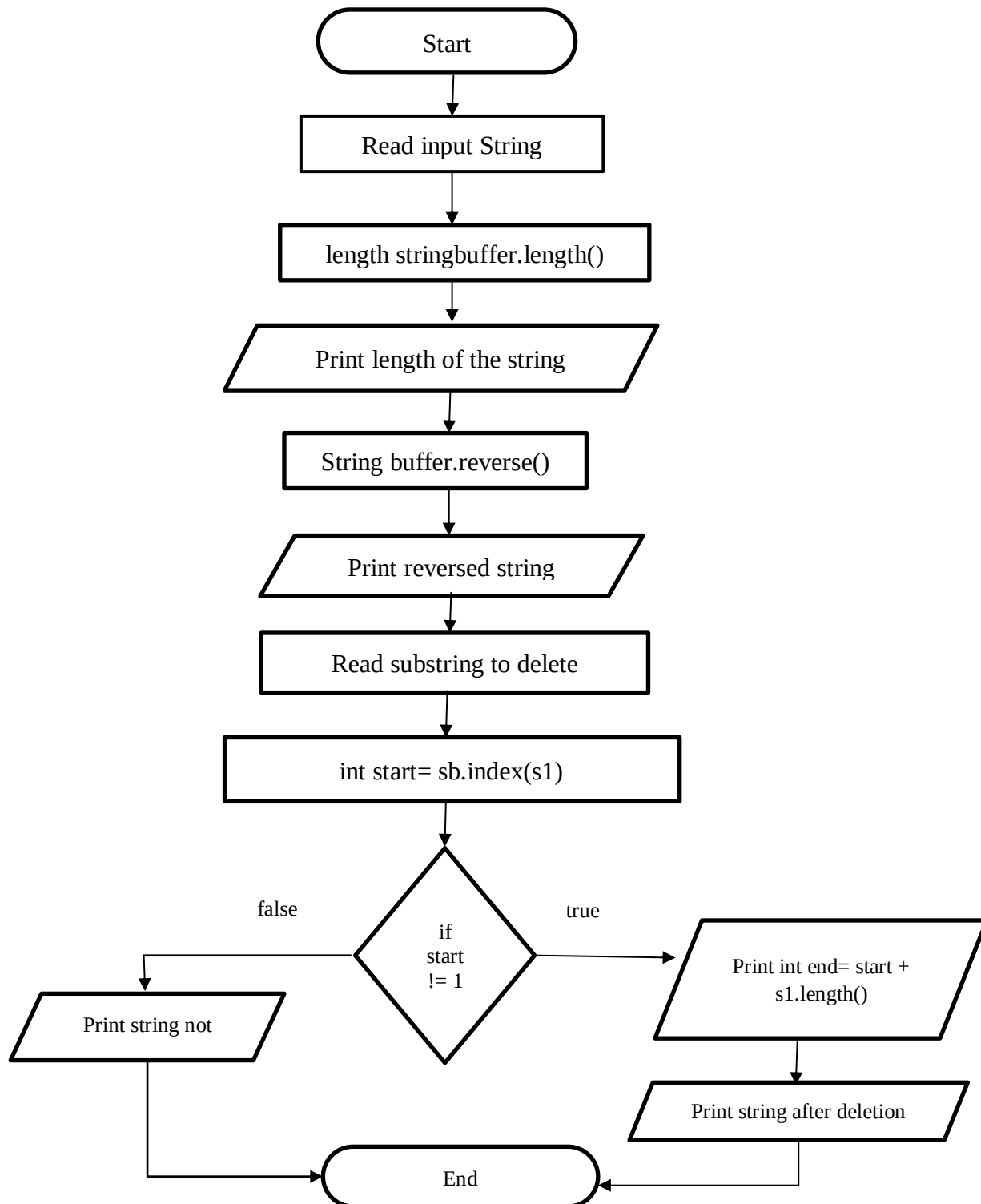
Step 10: Close Scanner

Step 11: End

CLASS DIAGRAM:



FLOWCHART:



PROGRAM:

```
import java.util.Scanner;

public class buffer {

    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.println("Enter a String: ");
        String input = s.nextLine();
        StringBuffer sb = new StringBuffer(input);
        int length = sb.length();
        System.out.println("Length of the string: " + length);
        sb.reverse();
        System.out.println("Reversed string: " + sb);
        sb = new StringBuffer(input);
        System.out.print("Enter a substring to delete: ");
        String s1 = s.nextLine();
        int start = sb.indexOf(s1);
        if (start != -1) {
            int end = start + s1.length();
            sb.delete(start, end);
            System.out.println("String after deleting the substring: " + sb);
        } else {
            System.out.println("Substring not found in the given string.");
        }
        s.close();
    }
}
```

OUTPUT:

Enter a String: welcome
Length of the string: 7
Reversed string: emoclew
Enter a substring to delete: wel
String after deleting the substring: come
Enter a String:
i can't
Length of the string: 7
Reversed string: t'nac i
Enter a substring to delete: 't
String after deleting the substring: i can

RESULT:

Thus the program has been executed successfully .

EX.NO : 8	A MULTI-THREADED APPLICATION USING THREE THREADS
DATE:	

AIM:

To create a Java program in which one thread generates a random number every 1 second. If the number is even, another thread prints its square. If the number is odd, third thread prints its cube.

ALGORITHM:

Step 1: Start the program.

Step 2: Create a thread to generate a random number.

Step 3: Generate a random integer every 1 second.

Step 4: Check whether the number is even or odd.

Step 5: If even, calculate and print the square.

Step 6: If odd, calculate and print the cube.

Step 7: Repeat the process.

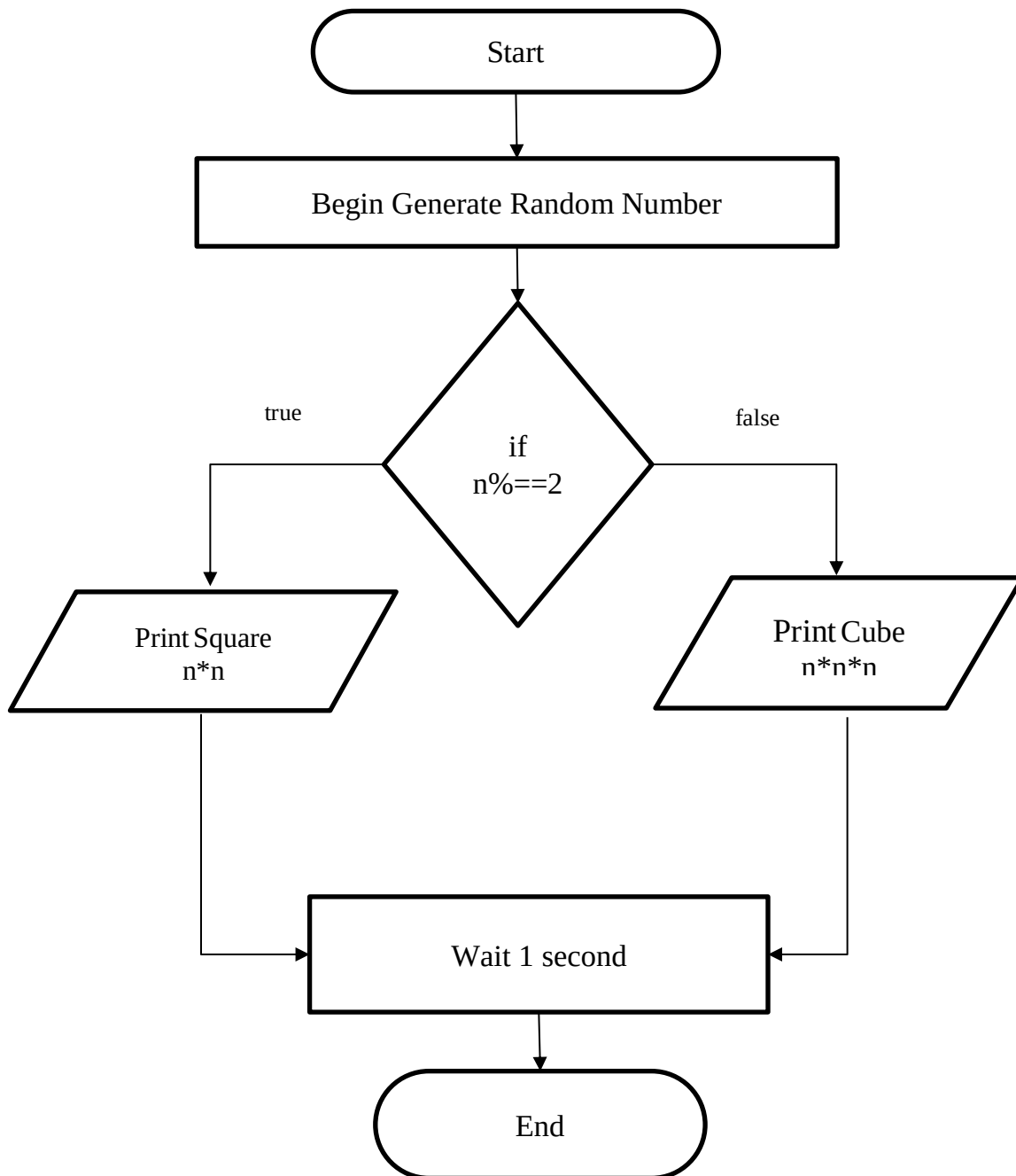
Step 8: Stop the program

CLASS DIAGRAM:

```
class SimpleMultiThread
class MyThread extends Thread

int random number, square, cube
```

FLOWCHART:



PROGRAM:

```
class MyThread extends Thread {
    public void run() {
        try {
            while (true) {
                int n = (int)(Math.random() * 10);
                System.out.println("Generated Number: " + n);
                if (n % 2 == 0)
                    System.out.println("Square: " + (n * n));
                else
                    System.out.println("Cube: " + (n * n * n));
                Thread.sleep(1000);
            }
        } catch (Exception e)
        {
            System.out.println(e);
        }
    }
}

public class SimpleMultiThread
{
    public static void main(String[] args)
    {
        MyThread t = new MyThread();
        t.start();
    }
}
```

OUTPUT:

Generated Number:7

Cube:343

Generated Number:0

Square:0

Generated Number:4

Square:16

Generated Number:3

Cube:27

Generated Number:3

Cube:27

Generated Number:0

Square:0

Generated Number:3

Cube:27

Generated Number:9

Cube:729

RESULT:

Thus the program has been executed successfully .

EX.NO : 9	ASYNCHRONOUS NUMBER PRINTING USING TWO THREADS
DATE:	

AIM:

To write a threading program which uses the same method asynchronously to print the numbers 1 to 10 using Thread1 and to print 90 to 100 using Thread2.

ALGORITHM:

Step 1: Start the program.

Step 2: Create a class PrintNumbers that extends Thread.

Step 3: Use a constructor to set the start and end numbers.

Step 4: In the run() method, print numbers from start to end.

Step 5: In main(), create **two thread objects** with different ranges:

Thread1 → 1 to 10

Thread2 → 90 to 100

Step 6: Set names for the threads.

Step 7: Start both threads using start().

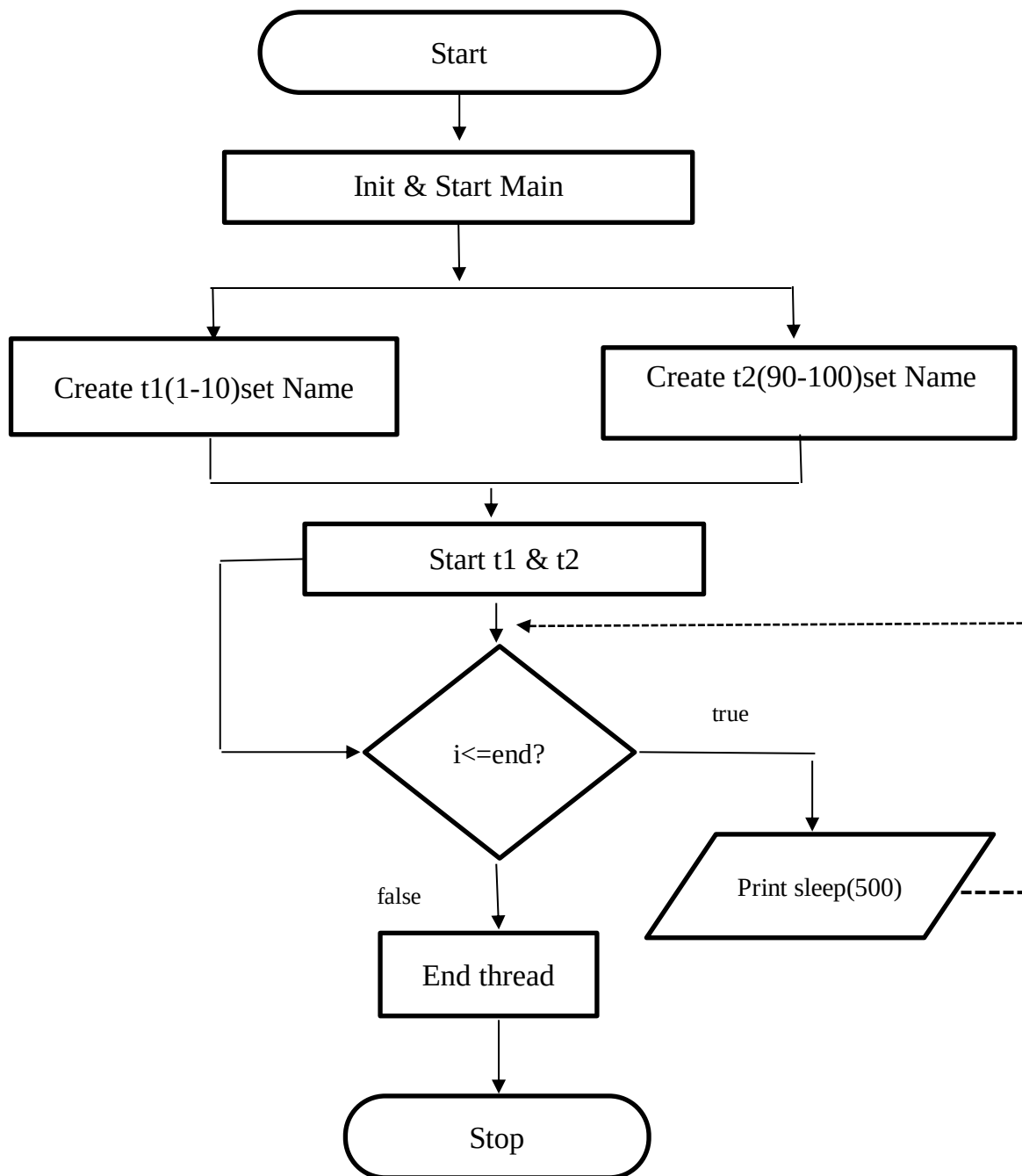
Step 8: Observe the numbers printed asynchronously.

Step 9: Stop the program.

CLASS DIAGRAM:

```
class ThreadRangeDemo
class PrintNumbers extends Thread
int start;
int end;
```

FLOWCHART:



PROGRAM CODING:

```
class PrintNumbers extends Thread {  
    int start, end;  
    PrintNumbers(int s, int e) {  
        start = s; end = e;  
    }  
    public void run() {  
        for(int i = start; i <= end; i++) {  
            System.out.println(Thread.currentThread().getName() + ": " + i);  
            try {  
                Thread.sleep(500);  
            }  
            catch(Exception e) {}  
        }  
    }  
}  
  
public class ThreadRangeDemo {  
    public static void main(String[] args) {  
        PrintNumbers t1 = new PrintNumbers(1, 10);  
        PrintNumbers t2 = new PrintNumbers(90, 100);  
        t1.setName("Thread1");  
        t2.setName("Thread2");  
        t1.start();  
        t2.start();  
    }  
}
```

OUTPUT:

```
Thread T2 : 90
Thread T1 : 1
Thread T2 : 91
Thread T1 : 2
Thread T1 : 3
Thread T2 : 92
Thread T1 : 4
Thread T2 : 93
Thread T1 : 5
Thread T2 : 94
Thread T2 : 95
Thread T1 : 6
Thread T1 : 7
Thread T2 : 96
Thread T2 : 97
Thread T1 : 8
Thread T2 : 98
Thread T1 : 9
Thread T2 : 99
Thread T1 : 10
Thread T2 : 100
```

RESULT:

Thus the program has been executed successfully .

EX.NO : 10	DEMONSTRATION OF COMMON JAVA EXCEPTIONS
DATE:	

AIM:

To write a Java program that demonstrates handling multiple exceptions including `ArithmeticException`, `NumberFormatException`, `ArrayIndexOutOfBoundsException`, and `NegativeArraySizeException`.

ALGORITHM:

Step 1 : Start the program.

Step 2 : Use a try-catch block to handle `ArithmeticException` by dividing a number by zero.

Step 3 : Use a try-catch block to handle `NumberFormatException` by converting an invalid string to integer.

Step 4 : Use a try-catch block to handle `ArrayIndexOutOfBoundsException` by accessing an invalid array index.

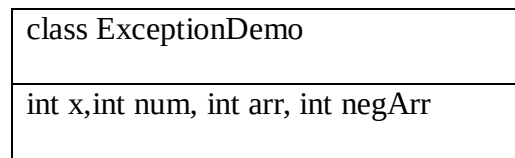
Step 5 : Use a try-catch block to handle `NegativeArraySizeException` by creating an array with negative size.

Step 6 : Print a message in each catch block describing the exception.

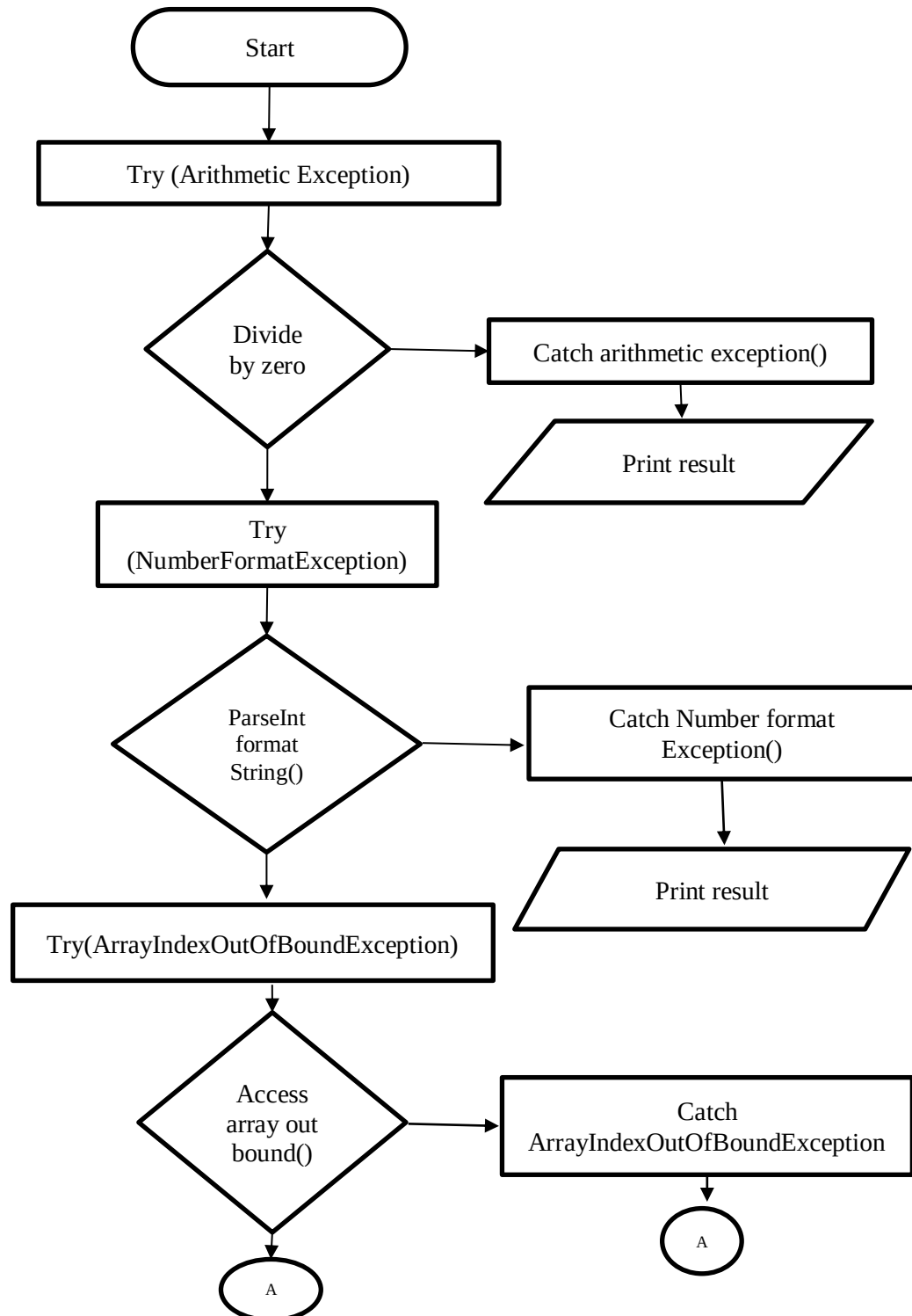
Step 7 : Continue program execution after handling exceptions.

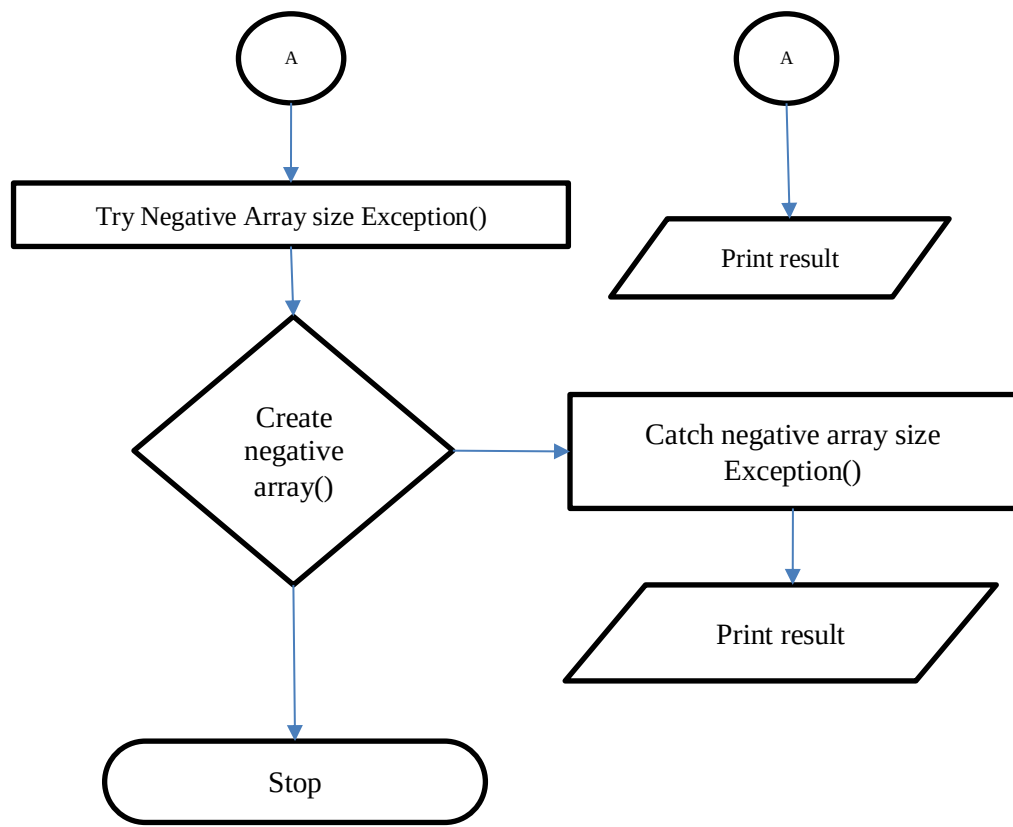
Step 8 : Stop the program.

CLASS DIAGRAM:



FLOWCHART:





PROGRAM:

```
public class ExceptionDemo {
    public static void main(String[] args) {
        System.out.println("=== Exception Handling Demo ===\n");

        // a) ArithmeticException
        try {
            int x = 10 / 0;
        } catch(ArithmeticException e) {
            System.out.println("1. ArithmeticException: Division by zero!");
        }

        // b) NumberFormatException
        try {
            int num = Integer.parseInt("abc");
        } catch(NumberFormatException e) {
            System.out.println("2. NumberFormatException: Invalid number format!");
        }

        // c) ArrayIndexOutOfBoundsException
        int[] arr = {1, 2, 3};
        try {
            System.out.println(arr[5]);
        } catch(ArrayIndexOutOfBoundsException e) {
            System.out.println("3. ArrayIndexOutOfBoundsException: Invalid index!");
        }

        // d) NegativeArraySizeException
        try {
            int[] negArr = new int[-5];
        } catch(NegativeArraySizeException e) {
            System.out.println("4. NegativeArraySize: Array size cannot be negative!");
        }

        System.out.println("\nAll exceptions handled successfully!");
    }
}
```

OUTPUT:

=== Exception Handling Demo ===

1. ArithmeticException: Division by zero!
2. NumberFormatException: Invalid number format!
3. ArrayIndexOutOfBoundsException: Invalid index!
4. NegativeArraySize: Array size cannot be negative!

All exceptions handled successfully!

RESULT:

Thus the program has been executed successfully .